

Channel Sounding Examples in src

What each initiator and reflector example does, how the pairs exchange measurements, and how the shared estimator turns CS step data into distance.

4

runnable role examples

2

execution models

1

shared estimator



1. Scope and coverage

This guide reads the C files directly under `src/`. The implementation in `src/libs/` is intentionally out of scope. The four files with a `main()` entry point form two matched device pairs; `distance_estimation.c` is their shared measurement-processing helper.

Coverage wording matters

There is one Twister-style manifest case in `sample.yaml`. At this snapshot, CMake compiles `cs_test_reflector.c` plus `distance_estimation.c`. The other three entry points are examples present in `src`, but they are not selected by the current build target. This document explains all four without implying that all four are currently automated test targets.

Source	Execution model	Role	Current selection
<code>cs_test_initiator.c</code>	Controller CS test API, then a temporary ACL/GATT exchange	Initiator; computes distance	Present, not selected
<code>cs_test_reflector.c</code>	Controller CS test API, then a temporary ACL/GATT exchange	Reflector; sends local steps	Selected by CMake
<code>connected_cs_initiator.c</code>	CS procedures inside an encrypted ACL connection	Initiator; configures and computes	Present, not selected
<code>connected_cs_reflector.c</code>	CS procedures inside an ACL connection	Reflector; returns local steps	Present, not selected
<code>distance_estimation.c</code>	Shared parser and estimator	Consumes both views	Selected helper

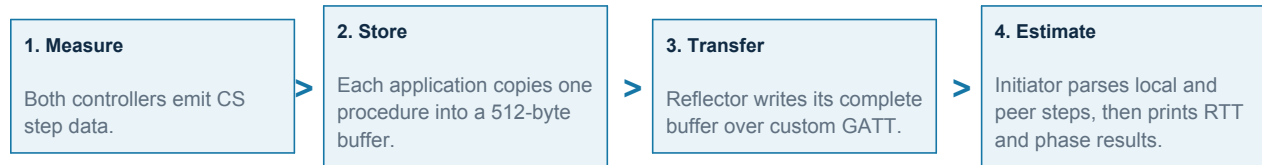
What the manifest checks

- The manifest defines `sample.bluetooth.channel_sounding.connected_cs.reflector`, uses the Bluetooth harness, and allows QEMU Cortex-M3 and x86.
- The manifest name says `connected_cs.reflector`, while CMake currently selects `cs_test_reflector.c`. Treat this as a naming/configuration mismatch worth reviewing.
- No unit-test source files appear directly under `src`; these are device examples exercised as firmware samples.

2. The common architecture

Channel Sounding produces local controller reports on both devices. A usable distance estimate needs measurements from both viewpoints. These examples therefore use two planes: the CS plane gathers radio measurements, and a custom GATT plane moves the reflector's raw step buffer to the initiator.

Two-device data path



The three CS step modes used

Mode	Purpose in these examples	How the code uses it	Result
Mode 0	Calibration / CS synchronization step	Skipped by the estimator; count helps validate a report and reconcile role-specific payload sizes	No direct distance output
Mode 1	Round-trip timing exchange	Pairs initiator ToA-ToD with reflector ToD-ToA; rejects bad AA check, missing RSSI, or missing timing	RTT-based distance
Mode 2	Phase-based ranging tones	Parses local and peer phase-correction terms; rejects low/unavailable tone quality	Phase-slope distance

Shared transport contract

- `common.h` defines one 128-bit service UUID and one characteristic UUID shared by all four role examples.
- `STEP_DATA_BUF_LEN` is 512 bytes, matching the maximum characteristic value the design expects to move.
- Each pair must use the same advertising name: `CS Test Sample` for test-mode, and `CS Sample` for connected-mode.
- The role pairs are not interchangeable. Their callback APIs, advertising names, and procedure setup differ.

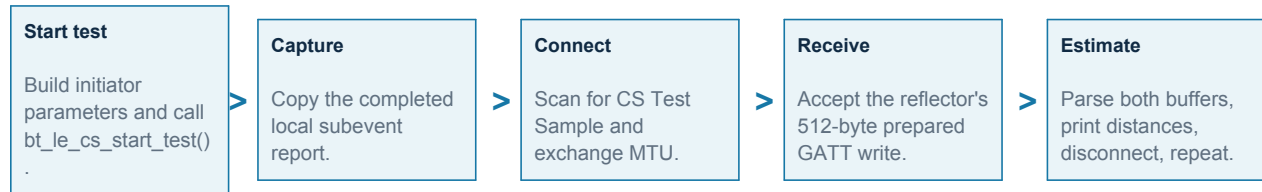
One procedure, one subevent

The connected examples deliberately constrain the channel map and subevent length so a procedure ends within one subevent. Both connected callbacks detect repeated procedure counters and decline to combine multiple subevents.

3. Test-mode initiator

`src/cs_test_initiator.c` uses Zephyr's controller CS test API without first creating a CS configuration over an ACL connection. It runs a standalone initiator-side measurement, then creates a normal BLE connection only to receive the reflector's report.

Control flow



Key behavior

- Registers `bt_le_cs_test_cb` callbacks. The subevent callback records the number of antenna paths, step count, step bytes, and completion status.
- Repeats the controller test until the report includes more than the three configured Mode 0 steps, ensuring at least one ranging step arrived.
- Stops the test explicitly, waits for the test-end callback, then starts active scanning for the paired reflector.
- Hosts the custom GATT service. The write callback requires offset 0, exactly 512 bytes, and copies data only on the Execute Write phase.
- Adjusts the peer buffer length because Mode 0 initiator and reflector records have different struct sizes, then calls `estimate_distance(..., INITIATOR)`.

Test parameter profile

Setting	Value	Meaning	Operational effect
Mode	Main 2 / sub 1	Phase ranging with RTT submode	Feeds both estimators
Mode 0 steps	3	Calibration/synchronization	Report must exceed this count
Main mode steps	8	Override config 2	Limited ranging payload
Channels	40 through 74	35 enabled channels	Controlled test sweep
Antenna	A1/B1	One tone antenna path per side	Simple pairing
RTT type	AA only	Access-address timing	Mode 1 timing input

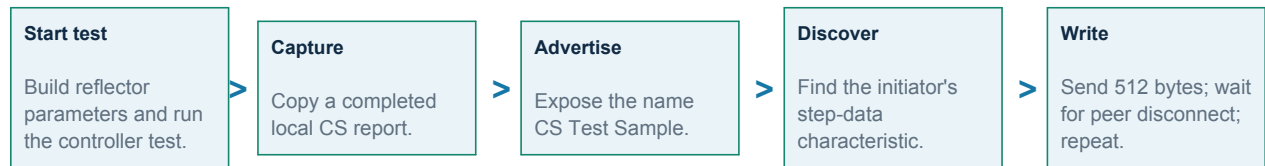
Pair with

`cs_test_reflector.c`. Flash both devices with matching test parameters; the initiator is the side that ultimately prints the distance.

4. Test-mode reflector

`src/cs_test_reflector.c` mirrors the standalone controller test as the reflector. After capturing a useful local report, it advertises, connects to the initiator, discovers the custom characteristic, and writes its raw buffer.

Control flow



Key behavior

- Uses the same shared `test_params_get()` profile as the test initiator, changing only the CS role to reflector.
- Requires more than the three Mode 0 steps before it proceeds to transport, matching the initiator's useful-report gate.
- Advertises connectably with the complete device name `CS Test Sample`.
- Acts as the GATT client: discovers the characteristic UUID and writes the entire 512-byte buffer.
- Does not estimate distance. Its job is to provide the peer half of the data and then wait for the initiator to terminate the connection.

Synchronization points

Semaphore	Released by	Main thread is waiting for
<code>sem_results_available</code>	Completed or aborted CS subevent	A captured controller report
<code>sem_test_complete</code>	CS test end callback	Safe transition out of test mode
<code>sem_connected</code>	BLE connected callback	An ACL link to the initiator
<code>sem_discovered</code>	Matching GATT characteristic	A write handle
<code>sem_disconnected</code>	BLE disconnected callback	Permission to begin the next cycle

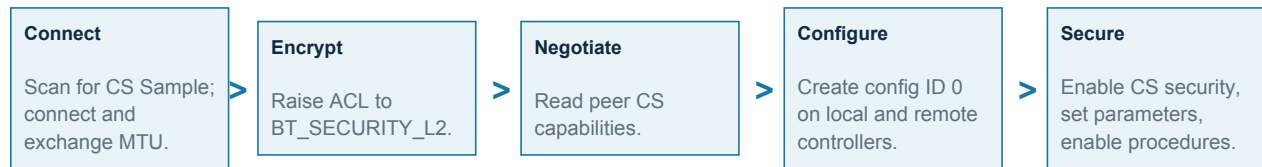
Current build target

The repository's current `CMakeLists.txt` compiles this entry point and `distance_estimation.c`. The estimator is linked even though this reflector file does not call it.

5. Connected initiator

`src/connected_cs_initiator.c` demonstrates the normal connection-oriented CS lifecycle. It owns orchestration: connect, encrypt, exchange capabilities, create a local-and-remote configuration, enable CS security, set procedure parameters, and enable repeating procedures.

Setup state machine



Configuration choices

Field	Configured value	Why it matters
Role	Initiator only	Makes this device the procedure orchestrator and estimator
Mode	Main 2 / sub 1	Combines phase-based and RTT measurements
Channel map	26 through 61, excluding spec-disallowed 23-25	Limits payload while retaining a frequency sweep
Main mode steps	2 to 10	Bounds steps per procedure
Procedure interval	100 to 100	Fixed repeating cadence in controller units
Subevent length	50,000 to 50,000 us	Forces channel exhaustion in the first subevent
Procedure count	0	Controller repeats until disabled

Steady-state loop

- Wait for the local procedure-complete callback, then wait for the reflector's prepared-write payload.
- Parse both buffers and print RTT-based and phase-slope-based estimates.
- Track `procedure_counter`; if a second subevent arrives for the same procedure, report that the sample cannot handle it.

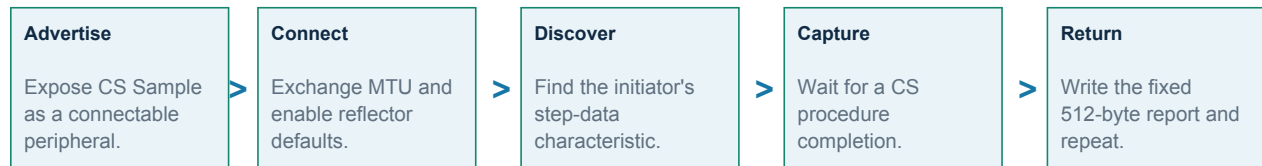
Why the 512-byte limit shapes radio configuration

The code comments explicitly choose channel-map repetition and a restricted map so the number of steps is controller-independent and the reflector can transfer the whole procedure with one 512-byte GATT value.

6. Connected reflector

`src/connected_cs_reflector.c` is the passive half of the connected pair. It advertises, accepts the initiator's ACL connection and remote CS configuration, captures each completed reflector report, and sends it back over GATT.

Control flow



Responsibility split

Concern	Initiator owns	Reflector owns
BLE role	Scanner / central	Advertiser / peripheral
CS setup	Encryption, capabilities, local-and-remote config, security, parameters, enable	Applies remote setup and reports completion through callbacks
Local report	Stores initiator steps and antenna-path count	Stores reflector steps
GATT	Characteristic server; accepts prepared write	Characteristic client; discovers and writes
Distance	Runs both estimators	Does not estimate

Important implementation details

- The reflector registers CS capability, configuration, security, procedure, and subevent callbacks. In the pair, the initiator triggers the setup operations.
- Discovery occurs once after connection. Each later procedure reuses the characteristic value handle.
- The write-complete semaphore gives backpressure: the next report is not sent until the current GATT write completes.
- As in the connected initiator, a repeated procedure counter signals an unsupported multiple-subevent procedure.

Pair with

`connected_cs_initiator.c`. Do not pair this example with a test-mode peer: the names and CS setup models intentionally differ.

7. Shared distance estimation

src/distance_estimation.c parses local steps first and peer steps second into shared arrays. The role argument determines which Mode 1 field belongs to the initiator and reflector, so the same code can assemble timing pairs from either viewpoint.

Phase-based ranging (Mode 2)

```
z = local_IQ * peer_IQ
theta = atan2(imag(z), real(z))
fit theta = a + b * frequency_MHz
distance_m = -b * c / (4 * pi * 1,000,000)
```

- For each usable tone, parse phase-correction I/Q values from both devices and form a complex product.
- Sort phase angles by channel frequency, unwrap jumps greater than pi, and fit a least-squares slope.
- The factor 4*pi reflects two-way phase accumulation; division by 1,000,000 converts the MHz-based slope to Hz-based units.
- Low-quality or unavailable tones are marked failed and excluded.

Round-trip timing (Mode 1)

```
one_way_time_units = (ToA-ToD_initiator - ToD-ToA_reflector) / 2
one_way_time_ns = one_way_time_units / 2
distance_m = one_way_time_ns * 0.299792458
```

- Combine the two devices' timing fields, halve the round-trip difference, then convert the controller's half-nanosecond units to nanoseconds.
- Multiply by the speed of light in meters per nanosecond.
- Reject Mode 1 samples with a failed access-address check, missing RSSI, or unavailable reflector timing.

Output semantics

Condition	Console behavior
Both estimates are zero	Prints that no reliable estimate could be computed
RTT estimate is nonzero	Prints Round-Trip Timing distance and parsed sample count
Phase estimate is nonzero	Prints Phase-Based Ranging distance and parsed tone count

These are example estimators

The implementation is intentionally compact: one-dimensional unwrapping, ordinary least squares, a cumulative mean, and quality gates. Production ranging normally adds calibration, antenna-delay compensation, robust outlier rejection, multipath handling, uncertainty, and bounds checks.

8. Choosing and pairing the examples

Question	Test-mode pair	Connected-mode pair
Best for	Direct controller test experiments with explicit test vectors	End-to-end Bluetooth CS procedure lifecycle
CS setup	bt_le_cs_start_test() / stop_test() on each device	Initiator configures CS across an encrypted ACL
BLE connection timing	Created after standalone CS capture	Exists before and during CS procedures
Advertising name	CS Test Sample	CS Sample
Initiator source	cs_test_initiator.c	connected_cs_initiator.c
Reflector source	cs_test_reflector.c	connected_cs_reflector.c
Result owner	Initiator	Initiator
Repeat behavior	Capture, connect, exchange, disconnect, repeat	Continuous procedures and writes on one connection

Build selection

Each example defines `main()`, so only one entry point can be compiled into a firmware image. Switch `target_sources(app PRIVATE ...)` to the desired role file and include `src/distance_estimation.c` only when the selected initiator calls it.

```
# Current snapshot
target_sources(app PRIVATE
src/cs_test_reflector.c
src/distance_estimation.c
)
```

Pairing checklist

- Select exactly one initiator and its matching reflector from the same execution model.
- Keep common .h, test parameters, buffer size, and advertising name aligned between the two images.
- Enable Bluetooth, GATT client, Channel Sounding, and - for test-mode - Channel Sounding Test in Kconfig.
- Ensure the negotiated ATT MTU and long-write behavior can carry the 512-byte characteristic value.
- Capture both consoles. The reflector confirms discovery/write progress; the initiator prints the final ranges.

Do not infer radio validity from a build-only QEMU run

The manifest uses a Bluetooth harness and QEMU platforms. That can exercise firmware integration and API paths, but real distance behavior requires CS-capable controllers and a physical two-device setup.

9. Limits, review notes, and source map

The examples are concise demonstrations. Their simplifying assumptions should be visible before they are reused as a measurement system.

Constraints visible in the code

- Fixed transport size: writers always send 512 bytes; receivers require exactly 512 bytes. The actual valid step length is tracked only on the initiator's local report.
- Single-subevent assumption: the connected pair reports and ignores a second subevent for the same procedure counter.
- Fixed storage: step buffers are 512 bytes and estimator sample arrays are 256 entries.
- Limited recovery: several asynchronous failures print an error but do not release the semaphore that the main thread is waiting on.
- Estimator accounting: the RTT cumulative mean divides by the loop index even when a failed sample is skipped, so invalid entries can bias the mean toward zero.
- Configuration mismatch: the manifest's connected-reflector test name does not match the CMake-selected test-mode reflector source.

Source map

File	Relevant content
src/cs_test_initiator.c	Standalone initiator CS test, GATT server, distance call
src/cs_test_reflector.c	Standalone reflector CS test, GATT discovery and write
src/connected_cs_initiator.c	Connected setup state machine, procedure control, distance call
src/connected_cs_reflector.c	Connected reflector callbacks and report forwarding
src/distance_estimation.c	Step parser, phase slope, RTT mean, console output
include/cs_test_params.h	Shared standalone test vector
include/common.h	UUIDs, name length, and 512-byte transport size
CMakeLists.txt	Selected application entry point
sample.yaml	Automated sample-test manifest
prj.conf	Bluetooth, Channel Sounding, MTU, console, and CRC options

Document basis

Generated from the repository working tree at
 /Users/yusein/Repositories/research/BLE_CS/cs_reflector on 27 Aug 2026. Library code under
 src/libs was excluded as requested.